```
!pip -q install transformers datasets sentence-transformers faiss-cpu accele
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.8/18.8 MB 73.6 MB/s eta 0:00:0
```

```
import pandas as pd
import numpy as np
import torch

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, f1_score

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
PyTorch: 2.11.0+cu128
CUDA available: True
```

```
import urllib.request
import zipfile
import os

url = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip

urllib.request.urlretrieve(url, "sms.zip")

with zipfile.ZipFile("sms.zip", "r") as z:
    z.extractall("data")

print(os.listdir("data"))
```

```
['readme', 'SMSSpamCollection']
```

```
df = pd.read_csv(
    "data/SMSSpamCollection",
    sep="\t",
    header=None,
    names=["label", "text"]
)

df["label"] = df["label"].map({
    "ham": 0,
    "spam": 1
})

print(df.head())
print(df["label"].value_counts())
```

```
   label                                               text
0      0  Go until jurong point, crazy.. Available only ...
1      0                      Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
```

```
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
label
0    4825
1     747
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df["label"]
)

print("Training samples:", len(train_df))
print("Testing samples :", len(test_df))

print("\nTrain distribution:")
print(train_df["label"].value_counts())

print("\nTest distribution:")
print(test_df["label"].value_counts())
```

```
Training samples: 4457
Testing samples : 1115

Train distribution:
label
0    3859
1     598
Name: count, dtype: int64

Test distribution:
label
0    966
1    149
Name: count, dtype: int64
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

vectorizer = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1, 2),
    sublinear_tf=True
)

X_train = vectorizer.fit_transform(train_df["text"])
X_test = vectorizer.transform(test_df["text"])

y_train = train_df["label"]
y_test = test_df["label"]
```

```
print("Train matrix:", X_train.shape)
print("Test matrix :", X_test.shape)

# something to compare against
# TF-IDF to logistic regression
```

```
Train matrix: (4457, 10000)
Test matrix : (1115, 10000)
```

```
lr = LogisticRegression(
    max_iter=1000,
    class_weight="balanced"
)

lr.fit(X_train, y_train)

baseline_pred = lr.predict(X_test)

print(classification_report(
    y_test,
    baseline_pred,
    target_names=["Legitimate", "Spam/Phishing"]
))
```

```
              precision    recall  f1-score   support

   Legitimate       0.99      0.99      0.99       966
Spam/Phishing       0.96      0.93      0.94       149

     accuracy                           0.98      1115
    macro avg       0.97      0.96      0.97      1115
 weighted avg       0.98      0.98      0.98      1115
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

baseline_results = {
    "Accuracy": accuracy_score(y_test, baseline_pred),
    "Precision": precision_score(y_test, baseline_pred),
    "Recall": recall_score(y_test, baseline_pred),
    "F1": f1_score(y_test, baseline_pred)
}

baseline_results
```

```
{'Accuracy': 0.9847533632286996,
 'Precision': 0.9583333333333334,
 'Recall': 0.9261744966442953,
 'F1': 0.9419795221843004}
```

RAG

```
!pip -q install sentence-transformers faiss-cpu
```

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
```

```
embedding_model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
```

```
modules.json: 100% 349/349 [00:00<00:00, 31.8kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 13.4kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 986kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.77kB/s]
config.json: 100% 612/612 [00:00<00:00, 51.7kB/s]
model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.75MB/s
model.safetensors: downloading bytes: 85.0MB, 8.05MB/s
Loading weights: 100% 103/103 [00:00<00:00, 3147.04it/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 32.2kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 14.6MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 27.5MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 11.2kB/s]
config.json: 100% 190/190 [00:00<00:00, 21.2kB/s]
```

```
knowledge_texts = train_df["text"].tolist()
knowledge_labels = train_df["label"].tolist()

knowledge_embeddings = embedding_model.encode(
    knowledge_texts,
    normalize_embeddings=True,
    show_progress_bar=True
)

knowledge_embeddings = np.array(
    knowledge_embeddings
).astype("float32")

print("Embedding shape:", knowledge_embeddings.shape)
```

```
Batches: 100% 140/140 [00:02<00:00, 100.39it/s]
Embedding shape: (4457, 384)
```

```
dimension = knowledge_embeddings.shape[1]

index = faiss.IndexFlatIP(dimension)

index.add(knowledge_embeddings)

print("Knowledge base size:", index.ntotal)
```

```
Knowledge base size: 4457
```

```
# RAG retrieval
def retrieve(query, k=5):

    query_embedding = embedding_model.encode(
        [query],
        normalize_embeddings=True
    )

    query_embedding = np.array(
        query_embedding
    ).astype("float32")

    scores, indices = index.search(
        query_embedding,
        k
    )

    results = []

    for score, idx in zip(scores[0], indices[0]):

        results.append({
            "text": knowledge_texts[idx],
            "label": knowledge_labels[idx],
            "similarity": float(score)
        })

    return results
```

```
query = "URGENT! Your bank account has been suspended. Click this link to ve

results = retrieve(query)

for i, result in enumerate(results):

    print(f"\nResult {i+1}")
    print("Similarity:", round(result["similarity"], 3))
    print("Label:", "SPAM" if result["label"] == 1 else "LEGIT")
    print("Text:", result["text"])
```

```

Result 1
Similarity: 0.569
Label: LEGIT
```

```
Text: Hiya , have u been paying money into my account? If so, thanks. Got a p

Result 2
Similarity: 0.525
Label: SPAM
Text: Your account has been credited with 500 FREE Text Messages. To activate

Result 3
Similarity: 0.511
Label: SPAM
Text: Urgent! call 09066350750 from your landline. Your complimentary 4* Ibiz

Result 4
Similarity: 0.511
Label: SPAM
Text: Urgent! call 09066350750 from your landline. Your complimentary 4* Ibiz

Result 5
Similarity: 0.507
Label: LEGIT
Text: My sort code is  and acc no is . The bank is natwest. Can you reply to
```

semantic similarity alone isn't enough to classify the message.

RAG classifier

```
def rag_predict(query, k=5):

    results = retrieve(query, k)

    # Weighted voting based on similarity
    spam_score = 0
    legit_score = 0

    for r in results:
        weight = max(r["similarity"], 0)

        if r["label"] == 1:
            spam_score += weight
        else:
            legit_score += weight

    total = spam_score + legit_score

    spam_probability = spam_score / total if total > 0 else 0

    prediction = int(spam_probability >= 0.5)

    return prediction, spam_probability, results
```

```
query = "URGENT! Your bank account has been suspended. Click this link to ve

prediction, probability, results = rag_predict(query)
```

```
print("Prediction:",
      "SPAM/PHISHING" if prediction else "LEGITIMATE")

print("Spam probability:", round(probability, 3))
```

```
Prediction: SPAM/PHISHING
Spam probability: 0.59
```

```
test_messages = [
    "URGENT! Your bank account has been suspended. Click this link to verify
    "Hey bro, are we still meeting at 5?",
    "Congratulations! You have won a free prize. Call now to claim.",
    "Please send me the notes from today's class.",
    "Your account will be closed unless you verify your information immediat
]

for message in test_messages:

    prediction, probability, _ = rag_predict(message)

    print("\nMessage:", message)
    print(
        "Prediction:",
        "SPAM/PHISHING" if prediction else "LEGITIMATE"
    )
    print("Probability:", round(probability, 3))
```

```
Message: URGENT! Your bank account has been suspended. Click this link to ver
Prediction: SPAM/PHISHING
Probability: 0.59

Message: Hey bro, are we still meeting at 5?
Prediction: LEGITIMATE
Probability: 0.0

Message: Congratulations! You have won a free prize. Call now to claim.
Prediction: SPAM/PHISHING
Probability: 1.0

Message: Please send me the notes from today's class.
Prediction: LEGITIMATE
Probability: 0.0

Message: Your account will be closed unless you verify your information immed
Prediction: SPAM/PHISHING
Probability: 0.795
```

```
!pip -q install transformers datasets accelerate
```

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
```

```
    TrainingArguments,
    Trainer
)
```

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

```
Loading weights: 100%                                    100/100 [00:00<00:00, 2069.22it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbe
Key                     | Status     |
------------------------+------------+-
vocab_transform.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
classifier.weight       | MISSING    |
pre_classifier.weight   | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
- MISSING:  those params were newly initialized because missing from the chec
```

```
from datasets import Dataset

train_hf = Dataset.from_pandas(
    train_df[["text", "label"]].reset_index(drop=True)
)

test_hf = Dataset.from_pandas(
    test_df[["text", "label"]].reset_index(drop=True)
)
```

```
def tokenize(batch):

    return tokenizer(
        batch["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

train_hf = train_hf.map(
    tokenize,
    batched=True
```

```
)

test_hf = test_hf.map(
    tokenize,
    batched=True
)
```

```
Map: 100%                                   4457/4457 [00:00<00:00, 5786.16 examples/s]
Map: 100%                                   1115/1115 [00:00<00:00, 3522.35 examples/s]
```

```
train_hf = train_hf.remove_columns(["text"])
test_hf = test_hf.remove_columns(["text"])

train_hf.set_format("torch")
test_hf.set_format("torch")
```

```
import torch

print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
```

```
True
Tesla T4
```

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
```

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("Model loaded successfully")
```

```
Loading weights: 100%                                    100/100 [00:00<00:00, 849.51it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbe
Key                     | Status     |
------------------------+------------+-
vocab_transform.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
classifier.weight       | MISSING    |
pre_classifier.weight   | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
- MISSING:  those params were newly initialized because missing from the chec
Model loaded successfully
```

```
print(train_hf)
print(test_hf)
```

```
Dataset({
    features: ['label', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 4457
})
Dataset({
    features: ['label', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 1115
})
```

```
training_args = TrainingArguments(
    output_dir="./phishing_distilbert",
    num_train_epochs=1,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    eval_strategy="epoch",
    save_strategy="no",
    logging_steps=50,
    report_to="none"
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_hf,
    eval_dataset=test_hf
)
```

```
trainer.train()
```

[279/279 00:51, Epoch 1/1]

Epoch	Training Loss	Validation Loss
1	0.029370	0.039785

```
from sklearn.metrics import classification_report, accuracy_score, precision_s

predictions = trainer.predict(test_hf)

pred_labels = np.argmax(
    predictions.predictions,
    axis=1
)

print(classification_report(
    y_test,
    pred_labels,
    target_names=["Legitimate", "Spam/Phishing"]
))
```

```
              precision    recall  f1-score   support

   Legitimate       0.99      1.00      1.00       966
Spam/Phishing       0.99      0.96      0.97       149

     accuracy                           0.99      1115
    macro avg       0.99      0.98      0.98      1115
 weighted avg       0.99      0.99      0.99      1115
```

```
distilbert_results = {
    "Accuracy": accuracy_score(y_test, pred_labels),
    "Precision": precision_score(y_test, pred_labels),
    "Recall": recall_score(y_test, pred_labels),
    "F1": f1_score(y_test, pred_labels)
}

distilbert_results
```

```
{'Accuracy': 0.9928251121076234,
 'Precision': 0.9862068965517241,
 'Recall': 0.959731543624161,
 'F1': 0.9727891156462585}
```